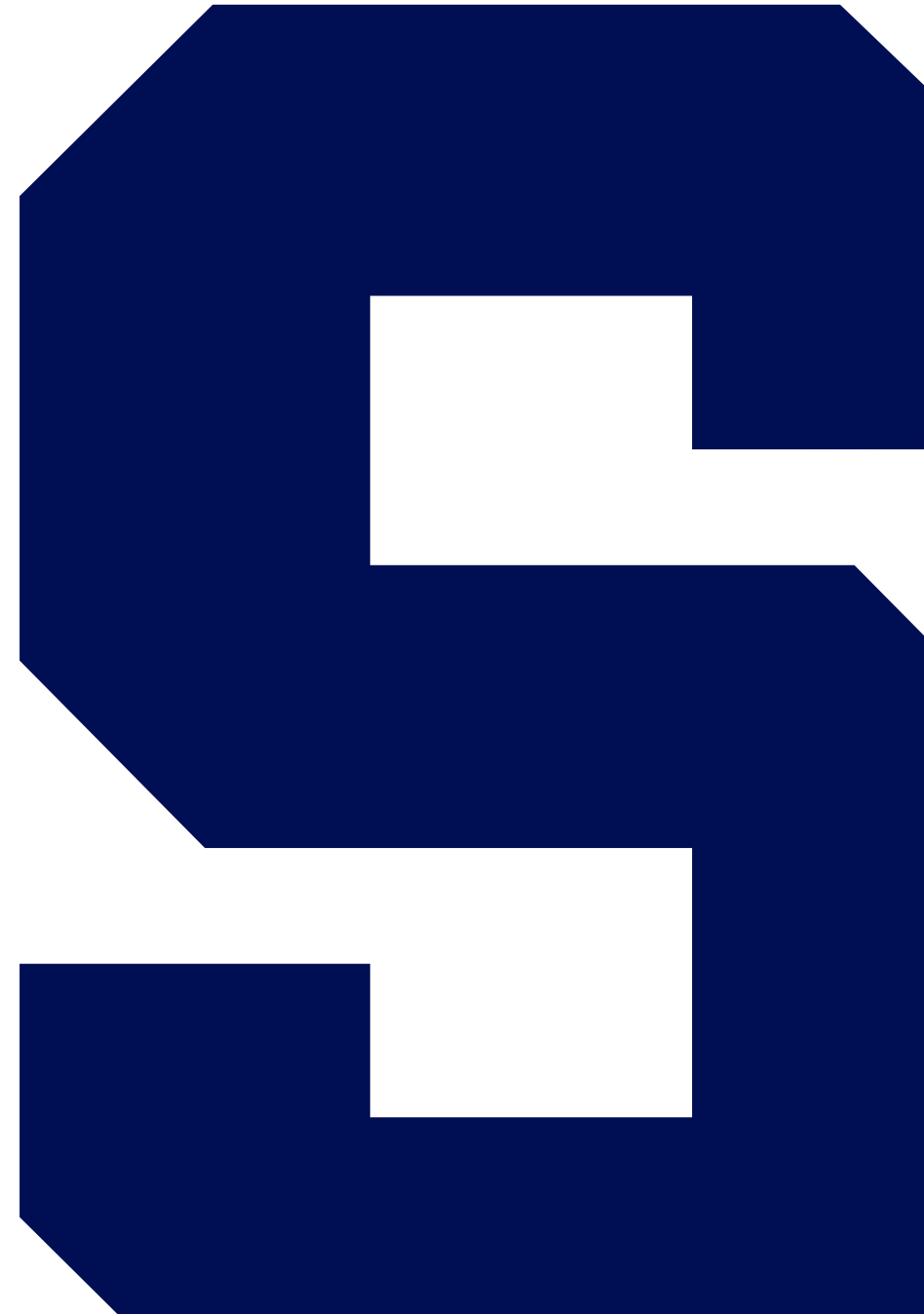


What is Deep Learning?



Deep Learning is a Subset of Machine Learning

- Deep Learning is a machine learning technique that enables computers to **learn by example**.
- Deep Learning is a part of a broader family of machine learning methods based on **artificial neural networks with representation learning**.
- Deep Learning models are loosely modeled on the information processing patterns found in the **human brain**.

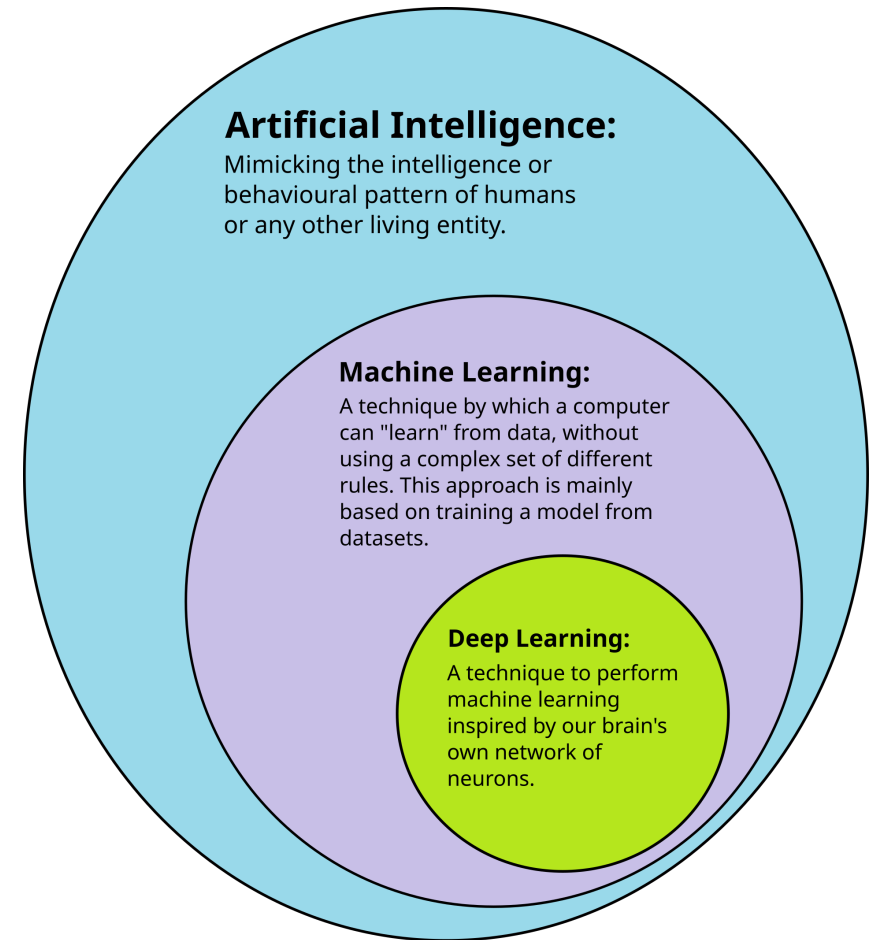


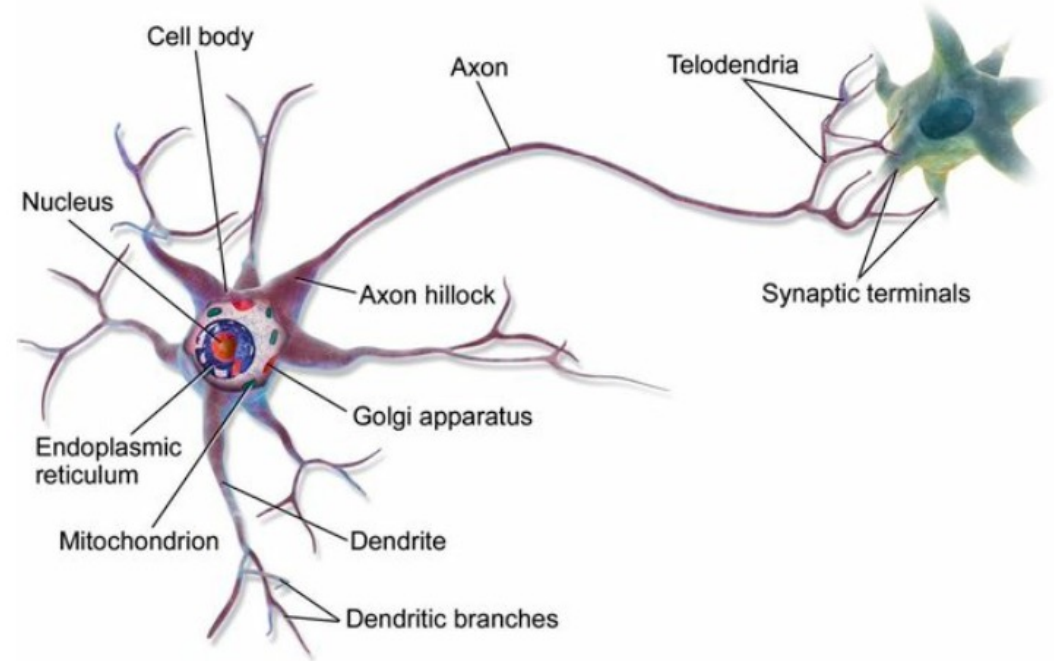
Image source: Wikipedia

Biological Neurons

Artificial Neural Networks are inspired by the human brain, which is composed of approximately 86 billion neurons.

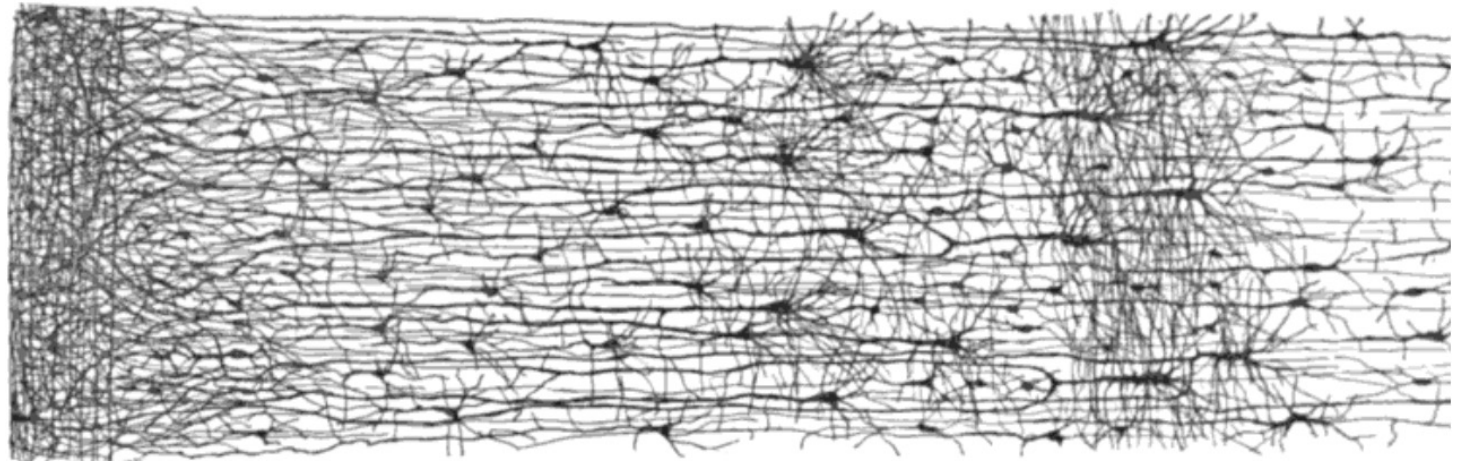
Structure of a Neuron: A neuron has a cell body with dendrites and an axon. The dendrites act like input channels, receiving signals from other neurons, while the axon acts like an output channel, sending signals to other neurons.

Synaptic Transmission: When a neuron receives a sufficient number of signals from its dendrites, it becomes **activated** and passes along an electrochemical signal through its axon.



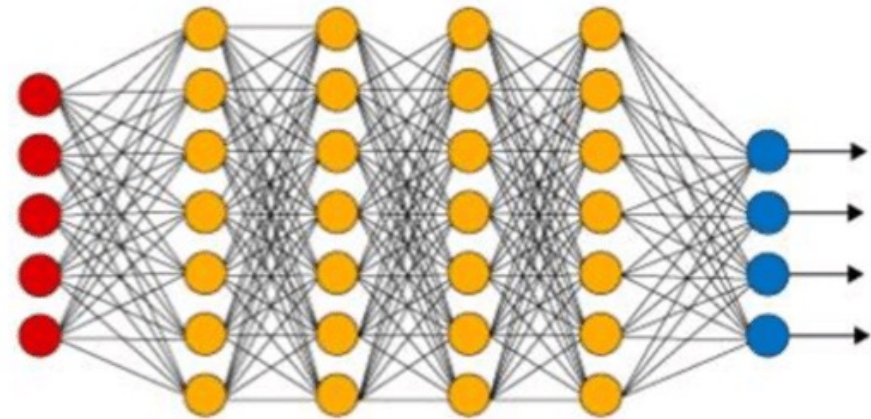
Networks of Neurons

- Individual neurons behave in a somewhat simple way.
- But neurons organized in a network of billions in multiple layers.
- Each neuron is connected to thousands of other neurons.
- Humans have 86 billion neurons
- African Elephants: 257 billion
- Pilot whales: 128 billion
- Chimpanzees: 28 billion
- Dogs: 2-4 billion
- Cats: 760 million
- Corn snake: 10 million
- Ants: 250 thousand
- Jellyfish: 5 thousand

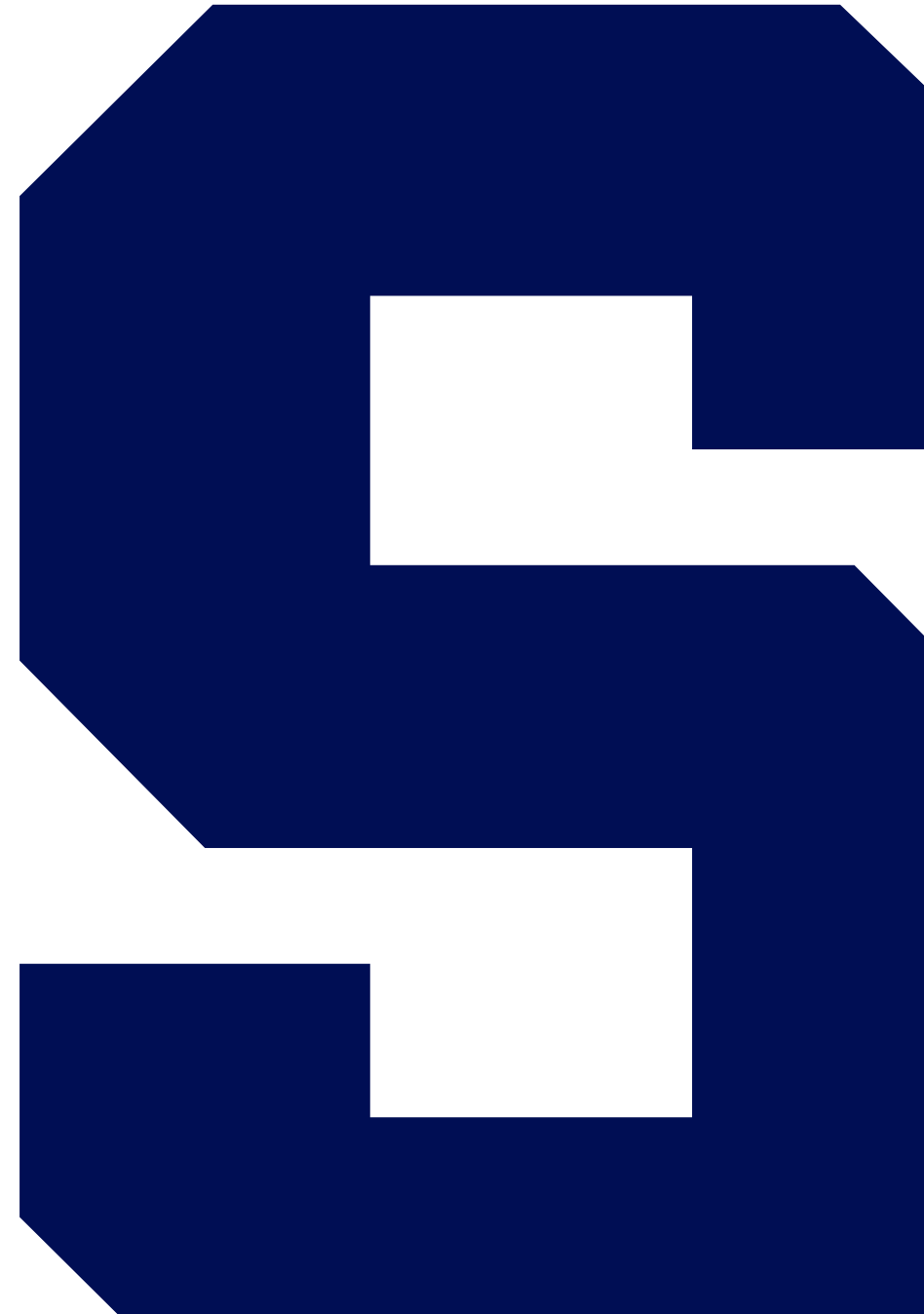


From Biology to Artificial Intelligence

- **Network of Neurons:** Each neuron can form thousands of links with other neurons, creating an incredibly complex network that is capable of a vast range of functions, from basic reflexes to advanced cognitive abilities.
- The neural networks in Deep Learning draw inspiration from this biological setup.
- The idea is to create **artificial "neurons"** that can connect and communicate with one another, forming networks that can learn to perform tasks without being explicitly programmed to do so.

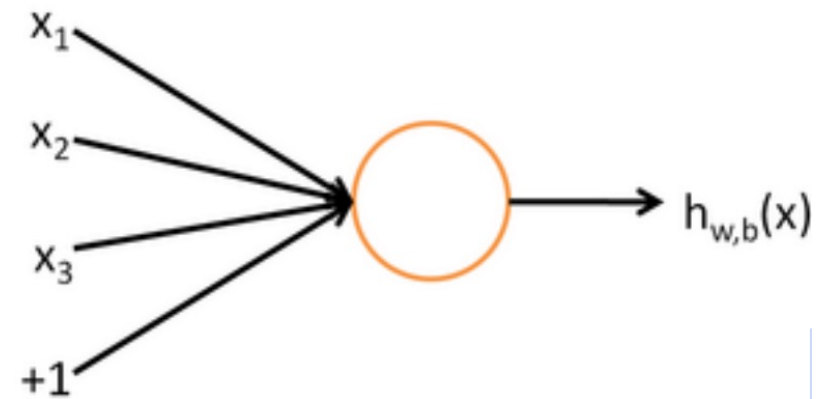


The Perceptron



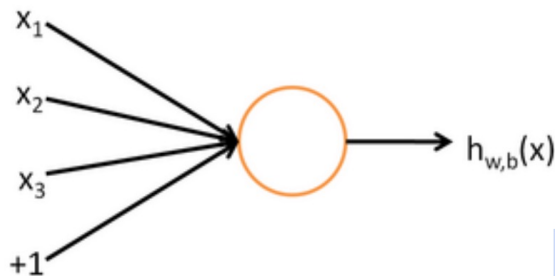
The Perceptron Model

- The **perceptron** is the simplest type of artificial neural network and can be considered the building block for more complex networks.
- Introduced by Frank Rosenblatt in 1957.
- Mimics a single neuron with multiple input signals and one output signal.
- **Operating principal:** computes a weighted sum of the inputs, adds a bias, and then passes this sum through an activation function to produce an output.



Learning: Weights and Bias

- **Weights:** In a perceptron, every input has an associated weight that represents its **relative importance**. These weights are adjustable **parameters** that the learning algorithm will learn.
- **Bias:** The bias is an additional parameter that allows the model to **shift** the activation function to the left or right, which helps the perceptron make better decisions.



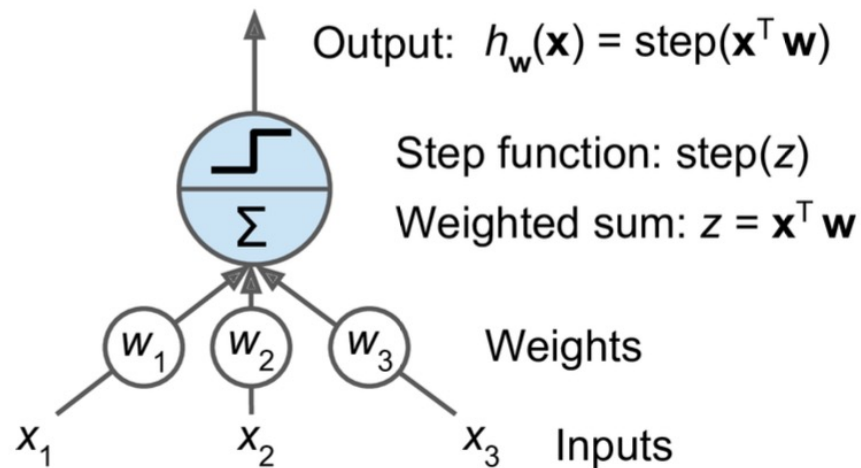
$$X = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = w^T x + b$$

Learning: Activation Function

Once the weighted sum of the inputs is computed, an **activation function** is applied to this sum. The simplest form of activation function is a step function. The most common step functions are the Heaviside step and sign functions:

$$\text{heaviside}(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$$

$$\text{sgn}(z) = \begin{cases} -1 & \text{if } z < 0 \\ 0 & \text{if } z = 0 \\ +1 & \text{if } z > 0 \end{cases}$$

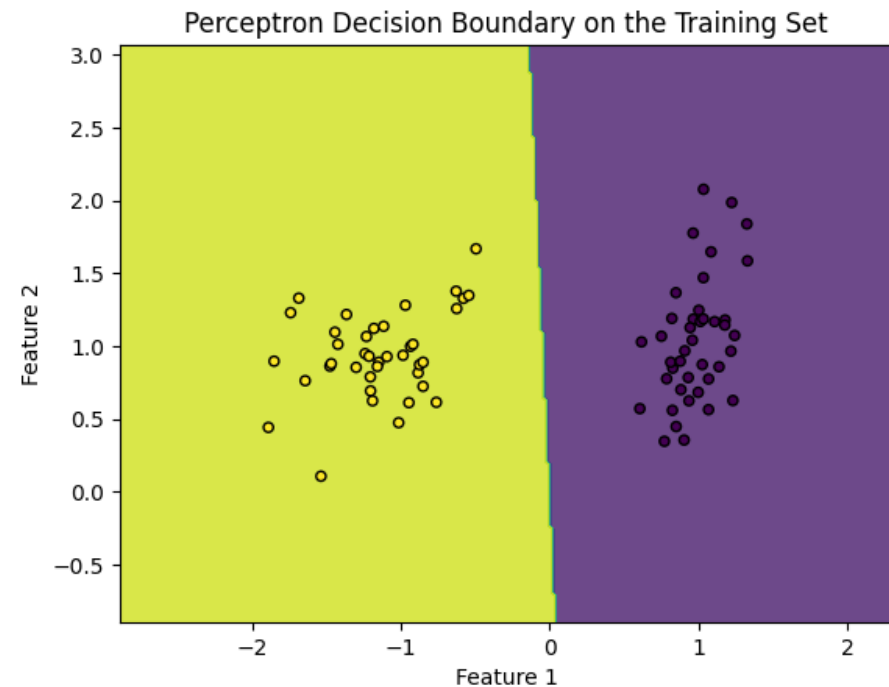


Learning: Feedback and Updates

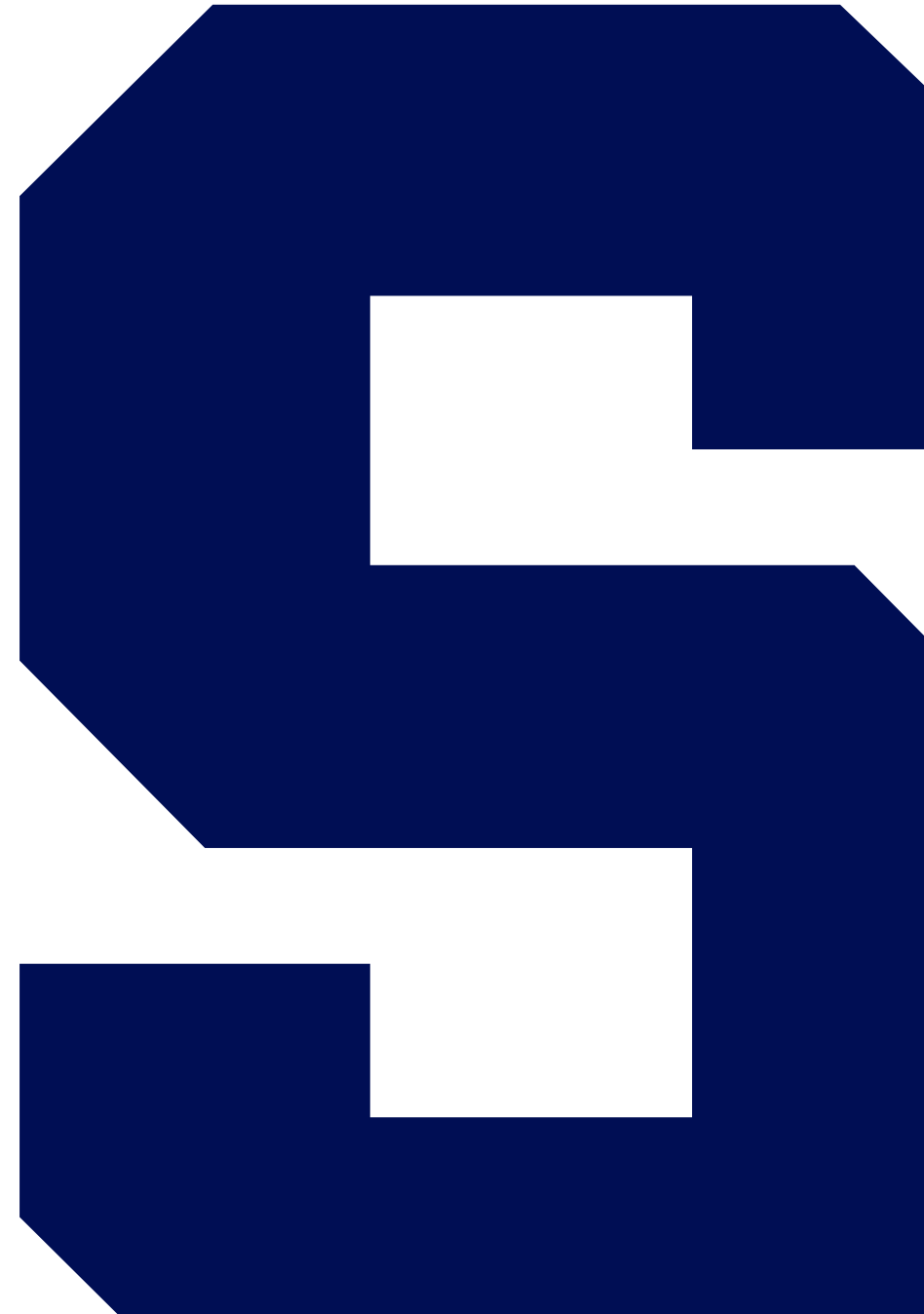
- During the training process, the perceptron receives inputs with known outputs.
 - It makes predictions by computing the weighted sum and applying the activation function.
 - If the prediction is wrong, the weights and bias are updated in such a way as to reduce the error in prediction.
 - This process is repeated for many iterations or until the model performs well.
-

Linear Separability and Limitations

- A single perceptron can only learn linearly separable patterns.
- This is a significant limitation because many real-world problems are not linearly separable.
- The publication of the book "Perceptrons" in 1969 pointed out that perceptrons are incapable of solving some simple problems (e.g., XOR).
- This led to a temporary decline in the popularity and funding of neural network research.
- The advent of MLPs and backpropagation brought about a resurgence in the field, leading to the deep learning revolution we are witnessing today.



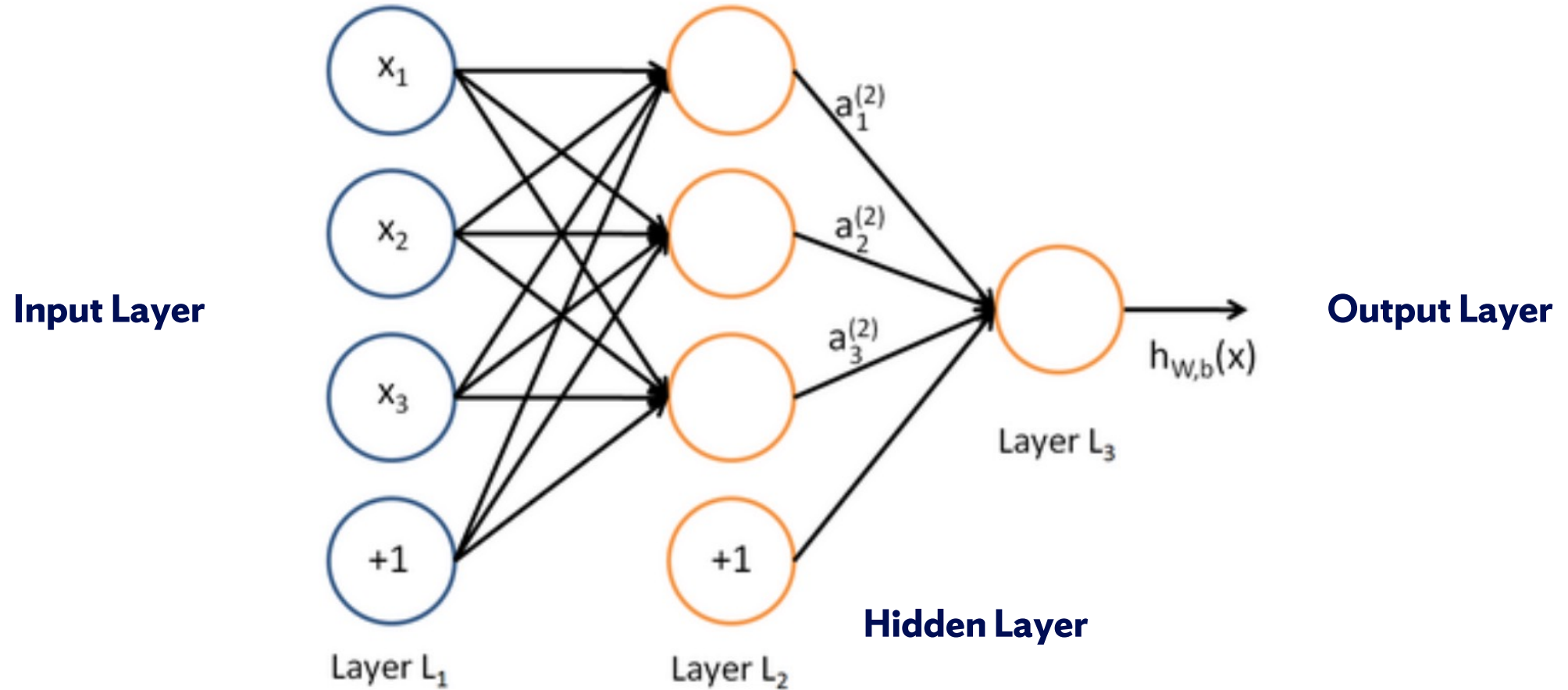
The Multilayer Perceptron (MLP)



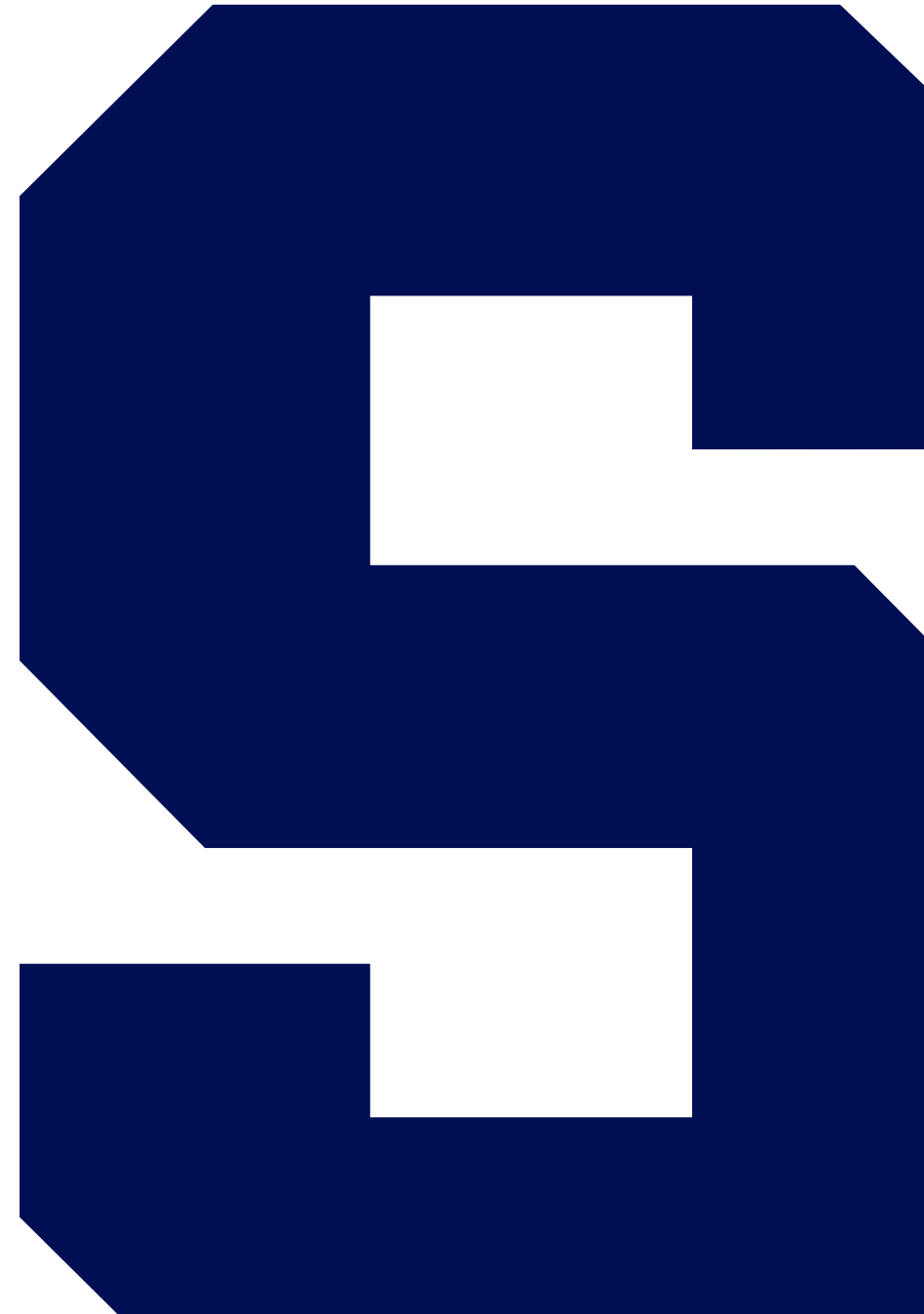
The Architecture of MLPs

- **Multilayer Perceptrons** (MLPs), also known as **feedforward neural networks**, are a class of deep neural networks with one or more hidden layers between the input and output layers.
 - Each layer is made up of a number of **interconnected nodes** or neurons, and each connection has a weight associated with it.
 - **Input Layer**: The first layer that receives the input signal to be processed.
 - **Hidden Layers**: One or more layers where the actual processing is done via a system of weighted connections. The number of hidden layers and the number of neurons in each hidden layer are parameters that can be adjusted depending on the complexity of the task.
 - **Output Layer**: The final layer that produces the output of the network.
-

A Multilayer Perceptron



Training a Multilayer Perceptron

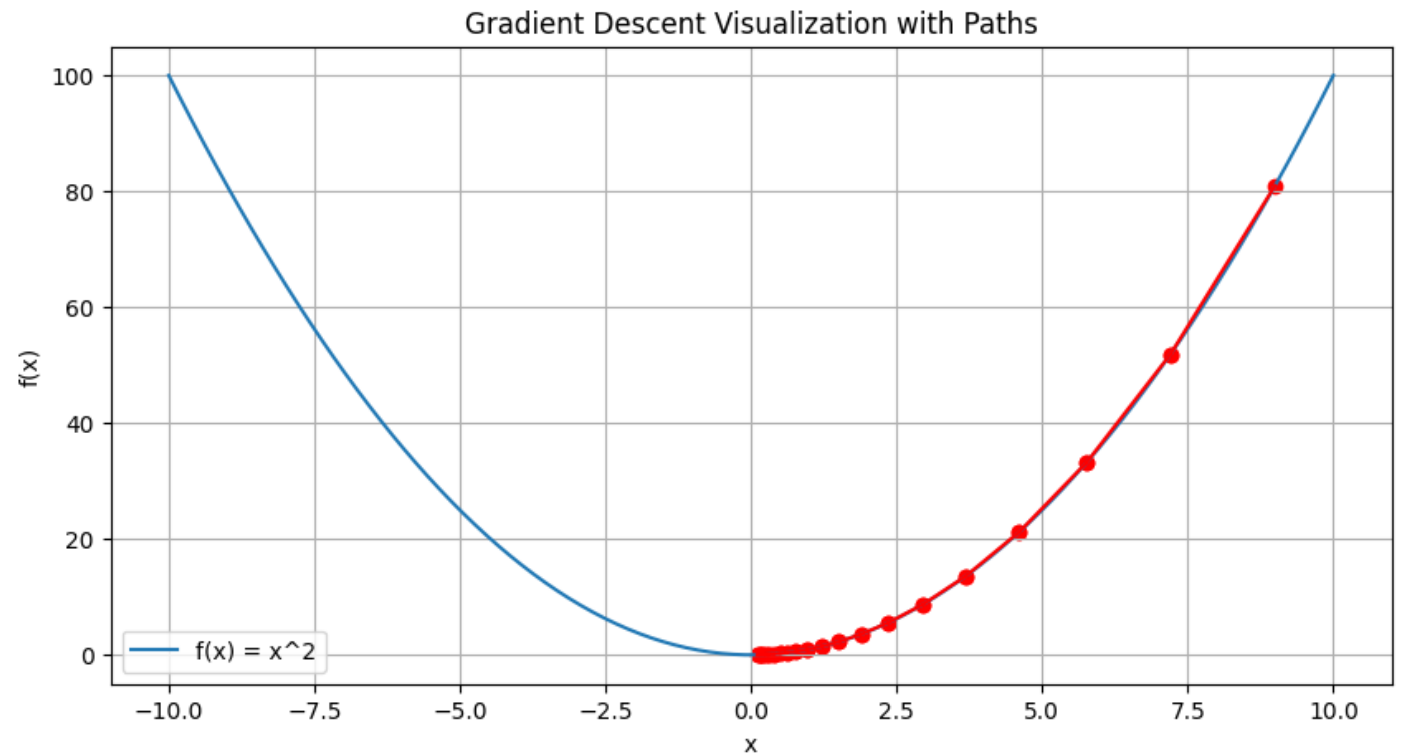


Gradient Descent

- **Gradient Descent** is the optimization algorithm used to minimize the cost function, which is a measure of how far off the network's predictions are from the actual values.
 - The process involves:
 - **Cost Function:** A function (like MSE for regression tasks) that measures the difference between the predicted values by the network and the true values.
 - **Learning Rate:** A hyperparameter that determines the size of the steps we take to reach a (local) minimum.
 - **Update Rule:** A rule to adjust the weights of the network in the opposite direction of the gradient of the cost function with respect to the weights.
-

Gradient Descent Example

- Imagine that we have a quadratic error function such as $f(x)=x^2$.
- The gradient of this error function is just the first derivative of the function $f(x)$.
- For each step, we move (slightly) in the direction of the negative gradient of the error function.



Training MLPs

- In 1970 Seppo Linnainmaa created the algorithm reverse-mode automatic differentiation.
 - In two passes through the network (one forward, one backwards) this algorithm is able to compute the gradients of the ANN error with regard to *all* model parameters.
 - These gradients can be used in a gradient descent step.
 - Combining this gradient step and reverse-mode automatic differentiation gives us **backpropagation**.
 - Backpropagation wasn't applied to ANNs until 1985!
-

Backpropagation Forward Pass

- Takes a **mini-batch** at a time, but goes through full training set multiple times.
 - Each pass through the full training set is an **epoch**.
 - Each mini-batch enters the network through the input layer.
 - Then passed to first hidden layer, and the next layer and so on.
 - Algorithm computes the output of every instance in the mini-batch.
 - This is the forward pass. Records all intermediary computations.
 - This is the same way that predictions are made (except we don't need to record intermediate values).
-

Backpropagation Backward Pass

- The algorithm measures the network's output error.
 - Computes how much each output bias and each connection to the output layer contributed to the error.
 - This is done via the chain rule from calculus.
 - Then it measures how much of these error contributions came from the layer below again using the chain rule, working backwards until it reaches the input layer.
 - Finally, the gradient descent step is used to tweak all the connection weights in the network using the error gradients it just computed.
-

Backpropagation Cont.

- Hidden layers connection weights are initialized randomly.
- Step function is replaced with one of many possible activation functions (step function has no gradient, which needed for gradient descent).
- Many popular activation functions:
 - ReLU (rectified linear unit) is commonly used: $ReLU(z) = \max(0, z)$

